FINAL PROJECT: MARIO BROS

Programming – Computer Science

Abstract

This document is the report for our final project, a clone of the “Game & Watch: Mario Bros.” game that we built with Python and Pyxel. It covers how we designed the classes using Object-Oriented Programming (OOP) concepts from class, the main logic that makes the game work, what we accomplished, a user manual, and our thoughts on the process. Our game has the main features of the original, where you control Mario and Luigi to move packages from conveyors to a truck.

Rodrigo Ferreiro Durio & Maximo Rivas

UC3M

TABLE OF CONTENTS:

1. Classes Design

We started designing the project by splitting everything into different classes, following the object-oriented structure we learned in the course. This approach was also part of the project requirements and it helped to keep the whole thing organised instead of ending up with one huge, unmanageable file.

The three main OOP principles that we actually used throughout the project were encapsulation, inheritance, and composition.

GameObject: This class is basically the base for almost everything else. Most objects in the game inherit from it so they automatically get their position through the x and y coordinates. The setters for these coordinates do checks before assigning anything, which keeps the data safe and follows the idea of encapsulation.

Encapsulation was the principle we focused on the most. In the lectures we were told how important it is to protect the internal data of a class, and we really tried to apply that. We didn't want other parts of the program changing attributes directly, so we used @property and @attribute.setter almost everywhere, just like we were shown in class. This also gave us a place to add checks. A good example is the Character class, where the setter for the floor always makes sure the value is a number, which can avoid crashes in the game.

We also used inheritance a lot to avoid repeating code:

- The Character class inherits from GameObject, and Mario and Luigi inherit from Character. This works well since Character has all the shared behaviour such as movement and handling states while Mario and Luigi only contain what makes them different, like controls and sprites.
- The Conveyor class inherits from Platform, giving it a base location while allowing it to specialize its functionality with moving packages.

Composition was another concept we used:

- **Game:** This class basically runs everything. It holds instances of almost all the other objects (Mario, Luigi, Truck, Boss, Doors...). It implements the essential state machine that controls the flow of the game (such as transitions between PLAYING, TRUCK_SEQUENCE, and GAME_OVER states).
- **PackageManager:** This class manages the entire collection of all Package objects. It handles the logic for spawning new packages periodically or when the number of packages is smaller than the minimum number of packages, updates their movement and handles what happens when a package is either received correctly or dropped.

2. The Main Logic

Getting the game to work involved a few important parts such as integrating the encapsulated logic within our classes, focusing heavily on accurate movement, package transfer, and game state synchronization.

The game operates as follows. Once a difficulty is selected and the factory is drawn, a first package appears on the machine conveyor. Mario can then move it to the main conveyors. Once there, Luigi and Mario can move it to the truck which will begin loading until eight packages have reached it. A truck sequence then starts and characters rest. Once 3 or 5 trucks have completed deliveries (depending on difficulty), one of the three lives can be restored (if the player still has three lives they do not receive another, and in CRAZY mode no lives are restored). One point is awarded for each package moved to the next conveyor and ten points are awarded when a truck goes for delivery.

Character Movement: Getting Mario and Luigi to move right was a good challenge. We wanted them to jump between floors rather than move pixel by pixel:

1. **Movement**: When a player presses a key (up or down), the character's floor variable is instantly modified by two units (for example, self.floor += 2). This integer value keeps the character logically positioned relative to the conveyor belts, so Mario can't be in a floor where he can't move a package.
2. **Positioning**: The separate internal method, update_y_pos, is called immediately after any floor change. This method translates the character's current floor index into the precise screen coordinate (y) using a constant list that contains the floor y positions (constants.FLOOR_Y_LEVELS). This is to make sure that the characters instantly "jump" to the correct vertical coordinate.

Package Movement and End Detection: Packages move autonomously across the conveyor belts, and their characteristics such as speed or direction derive from the assigned Conveyor object:

1. **Movement**: The Package's update method continuously increments or decrements its x coordinate using the speed and direction properties of the specific Conveyor it is on.
2. **End Detection**: A validation logic checks if the package has exceeded the conveyor's x limit. This checks the package's physical width and a defined overhang limit (constants.OVERHANG_LIMIT). When the package successfully exceeds this limit, it returns a specific status (PKG_STATUS_REACHED_END) to the PackageManager.

Collision Detection (Transfer Logic): For the "collisions" mechanic, which for us simply meant detecting when a package was caught, we did not implement complex mathematical calculations to determine if sprites were touching. Instead, we deployed a simpler, logical check implemented in the can_receive_package method:

1. The core logic ensures that the current package's level parity (even or odd floor index) matches the Character's permanently assigned initial floor parity (Mario for even, Luigi for odd).
2. It confirms that the Character is physically standing on the exact floor corresponding to the package's level.
3. **Success**: If both conditions are met, the Package is flagged for successful transfer to the next conveyor, awarding the player one point.
4. **Failure Trigger**: If the check fails, the package immediately triggers its falling state via the Package.fall method, starting a falling animation. This records a failure and triggers the Boss sequence via the Game.initiate_punishment method.

3. Work Carried Out

We have completed a fully playable game and are confident in its quality. It includes all the main features expected.

- **A Main Menu:** You can pick the difficulty (EASY, MEDIUM, EXTREME, CRAZY).
- **The Main Game:** You can move the characters, move packages, and load the truck.
- **Scoring:** You get points for doing things right.
- **Failures:** You can only fail 3 times before the game is over.
- **Difficulty Changes:** The game gets harder not just based on the difficulty selected, but also as your score goes up.
- **Animations:** We put in little “cutscenes” for when the truck is full or when the boss comes out. The game pauses so you can watch.
- **Game Over Screen:** It shows your final score and lets you go back to the menu.

Unimplemented Parts:
The game loop is solid, so we didn't leave anything half-finished. But if we had more time, it would have been cool to add sound or maybe make the boss do more than just yell at you. A high-score table would have been a nice thing too.

4. User Manual

The goal is to move packages from the factory's conveyor belts to the delivery truck making sure they don’t fall.

The first thing you have to do is to open main.py and run the code. The main menu will appear.

Controls (switched in crazy mode):

- **Luigi (Left Side):**
    - **W Key:** Move Up

    - **S Key:** Move Down
- **Mario (Right Side):**
    - **Up Arrow Key:** Move Up
    - **Down Arrow Key:** Move Down
- **Menu Navigation:** Use numbers 1-4 to select the desired difficulty.
- **Game Over:** Press space to return to the main menu.

5. Our Conclusions

Summary

Overall, we think the project was a success. We made a fun game that actually works, and we used all the OOP stuff we talked about in class. Breaking the problem down into classes made a huge difference and kept the project from becoming a total mess.

Main Problems We Ran Into

We ran into some problems:

1. **The platform.py Mistake:** We named a file platform.py because it had our platform code in it. But when we ran the game, it crashed with a weird import error. It turned out Python has its own library with that exact name, and it was getting confused. We renamed our file to game_platform.py and it got fixed immediately.

2. **Stopping the Players During Animations:** Another thing that was tricky was the game states. When the truck was driving off, you could still move Mario and Luigi around, and it just looked weird and buggy. We fixed this by building a state machine for the whole game in the Game class. We set up states like PLAYING and TRUCK_SEQUENCE. Then the main update loop would only run the logic for whatever state the game was in. So if the state was TRUCK_SEQUENCE, it just wouldn't check for player input. It was a simple idea but it made the whole game feel much more solid.

Despite these challenges, we were able to complete all requirements without significant complications.

Personal Comments

This project was a great way to actually use the OOP ideas from the lectures. It's different hearing about encapsulation than seeing how it helps keep our code clean. Making the PackageManager to handle all the package logic was probably one of our best decisions. It took a lot of messy code out of the other classes and put it all in one place. Before that, our Package class was extremely long.